SVKM's NMIMS
Mukesh Patel School of Technology Management & Engineering
A.Y. 2025 - 26
Course: Database Management Systems

Project Report

Program	BTech CSEDS 311	
Semester	Semester IV	
Name of the Project:	RaktSetu — Blood Bridge Management System	
Details of Project Members		
Batch: B2	Roll No.	Name:
	L056	Yash Shah
	L069	Varshita Vaan
	L074	Mahak Wadhawan
Date of Submission: 20/04/2026		

Contribution of each project Members:

Roll No.	Name:	Contribution
L056	Yash Shah	Database Design, Tables, Seed Data, SQL Queries
L069	Varshita Vaan	Flask Backend Connections, ER Diagram, SQL Queries
L074	Mahak Wadhawan	UI Frontend (HTML/CSS/JS), SQL Queries

Github link of your project: [mahak2604/DBMS_RaktSetu_Project: Blood Inventory Management System]

B Tech Computer Science and Engineering (Data Science)

-311 Program Semester – IV

Academic Year 2025-26

Database Management Systems

RaktSetu — Blood Bridge Management System

Project Report

Prepared by

Roll No.	Name
L074	Mahak Wadhawan
L056	Yash Shah
L069	Varshita Vaan

Faculty In charge

Dr. Dhirendra Mishra

SVKM's NMIMS

Mukesh Patel School of Technology Management and Engineering

Vile Parle West Mumbai-56

Table of Contents

Sr no.	Topic	Page no.
1	Problem statement	4
2	Components of Database Design	5
3	Entity Relationship Diagram	8
4	Relational Model	9
5	Normalization	10
6	SQL Queries	12
7	Project Demonstration	18
8	Self-Learning Beyond Classroom	25
9	Learning from the Project	26
10	Challenges Faced	27
11	Conclusion	28

I. Problem Statement

India faces a chronic shortage of safe, readily available blood across its hospitals and blood banks. Every two seconds, someone in India requires blood for surgery, accident recovery, childbirth complications, or treatment of diseases such as thalassemia and sickle-cell anaemia. The challenge is not merely one of supply it is one of connection. Donors exist, hospitals exist, blood banks exist, but there is no unified, real-time system to bridge them efficiently.

RaktSetu (meaning "Blood Bridge" in Sanskrit) is a web-based Blood Bridge Management System designed to address this gap. The system digitises and streamlines the entire blood lifecycle: from donor registration and blood collection through inventory management and finally to hospital blood requests and fulfilment.

Key Problems Addressed:

1. **Donor Discovery:** Healthcare workers and patients struggle to find compatible donors quickly, especially for rare blood groups such as AB− and O−. RaktSetu maintains a searchable donor registry filterable by blood group and city.
2. **Inventory Opacity:** Blood banks often lack real-time visibility into stock levels, leading to wastage from expired units or shortfalls during emergencies. RaktSetu maintains a live blood inventory table with expiry tracking.
3. **Request Management Delays:** Hospitals currently rely on phone calls and manual paperwork to raise blood requests and track their status. RaktSetu provides a structured digital workflow with priority levels (CRITICAL, HIGH, NORMAL) and status transitions (PENDING → APPROVED → FULFILLED).
4. **Data Fragmentation:** Donor history, hospital records, inventory, and requests are managed in silos. RaktSetu consolidates all entities into a single relational database with enforced referential integrity.

Scope of the System:

- Register, search, and delete blood donors
- Track blood inventory by blood group with expiry date monitoring
- Log and manage hospital blood requests with priority classification
- Provide an administrative SQL console for ad-hoc queries
- Expose all data through a REST API consumed by a responsive web frontend

The system is built using **MySQL** as the relational database, **Python (Flask)** as the backend REST API server, and **HTML/CSS/JavaScript** as the frontend interface all running locally in a three-tier architecture.

II. Components of Database Design

Entities and Their Attributes

1. DONORS Represents individuals who donate blood to the system.

Attribute	Type	Description
donor_id (PK)	VARCHAR(10)	Unique identifier, e.g., D001
name	VARCHAR(100)	Full name of the donor
age	INT	Age (constrained 18–65)
blood_group	VARCHAR(5)	ABO + Rh type, e.g., O+, AB−
phone	VARCHAR(15)	Contact number
city	VARCHAR(50)	City of residence
last_donated	DATE	Date of most recent donation

2. HOSPITALS Represents healthcare institutions that raise blood requests.

Attribute	Type	Description
hospital_id (PK)	VARCHAR(10)	Unique identifier, e.g., H001
name	VARCHAR(100)	Name of the hospital
city	VARCHAR(50)	City where the hospital is located
contact	VARCHAR(15)	Contact number

3. BLOOD_INVENTORY Tracks blood units available in the blood bank.

Attribute	Type	Description
inv_id (PK)	VARCHAR(10)	Unique identifier, e.g., I001
blood_group	VARCHAR(5)	Blood group of the stored units
units	INT	Number of units available
expiry_date	DATE	Date by which blood must be used
received_date	DATE	Date blood was received
donor_id (FK)	VARCHAR(10)	References DONORS(donor_id)

4. BLOOD_REQUESTS Represents requests raised by hospitals for blood units.

Attribute	Type	Description
req_id (PK)	VARCHAR(10)	Unique identifier, e.g., R001
hospital_id (FK)	VARCHAR(10)	References HOSPITALS(hospital_id)
patient	VARCHAR(100)	Name of the patient needing blood
blood_group	VARCHAR(5)	Required blood group
units	INT	Units requested
priority	ENUM	CRITICAL / HIGH / NORMAL
status	ENUM	PENDING / APPROVED / FULFILLED
contact	VARCHAR(15)	Contact number for the request
request_date	DATE	Date of the request

Relationships Among Entities

1. DONORS → BLOOD_INVENTORY (One-to-Many) A single donor can contribute blood multiple times, resulting in multiple inventory entries. However, each inventory record is linked to at most one donor. Cardinality: 1 (DONORS) to Many (BLOOD_INVENTORY). Participation: Total on BLOOD_INVENTORY side (every inventory entry has an associated donor or NULL if donor is deleted); partial on DONORS side. Foreign key: donor_id in BLOOD_INVENTORY references donor_id in DONORS with ON DELETE SET NULL.

2. HOSPITALS → BLOOD_REQUESTS (One-to-Many) A hospital can raise multiple blood requests over time, but each request is placed by exactly one hospital. Cardinality: 1 (HOSPITALS) to Many (BLOOD_REQUESTS). Participation: Partial on both sides — not every hospital has raised a request, and a request can exist without a hospital if the hospital record is deleted. Foreign key: hospital_id in BLOOD_REQUESTS references hospital_id in HOSPITALS with ON DELETE SET NULL.

III. Entity Relationship Diagram

ER Diagram Description:

- **DONORS** has a one-to-many relationship labelled *"donates blood to"* with **BLOOD_INVENTORY**. The relationship is partial on the DONORS side (a donor may not have donated yet) and total on the BLOOD_INVENTORY side (every blood bag came from a donor).

- **HOSPITALS** has a one-to-many relationship labelled *"raises"* with **BLOOD_REQUESTS**. Both sides have partial participation since hospital data can exist without any requests, and (via SET NULL) requests can exist without a linked hospital.

- **BLOOD_REQUESTS** and **BLOOD_INVENTORY** are indirectly connected through the blood group attribute — when a request is approved, the system operator fulfils it by dispensing matching units from inventory.

IV. Relational Model

Converting the ER diagram to the relational model yields the following four tables. Primary keys are underlined and foreign keys are marked with FK.

1. DONORS(**donor_id**, name, age, blood_group, phone, city, last_donated)

2. HOSPITALS(**hospital_id**, name, city, contact)

3. BLOOD_INVENTORY(**inv_id**, blood_group, units, expiry_date, received_date, donor_id_FK)

4. BLOOD_REQUESTS(**req_id**, hospital_id_FK, patient, blood_group, units, priority, status, contact, request_date)

Referential Integrity Constraints

- BLOOD_INVENTORY.donor_id → DONORS.donor_id (ON DELETE SET NULL)
- BLOOD_REQUESTS.hospital_id → HOSPITALS.hospital_id (ON DELETE SET NULL)

Indexes Created for Performance

- idx_donor_bg on DONORS(blood_group) — speeds up blood-group-based donor searches
- idx_inv_bg on BLOOD_INVENTORY(blood_group) — speeds up inventory lookups by blood group
- idx_req_status on BLOOD_REQUESTS(status) — speeds up filtering pending/approved requests

V. Normalization

Donors Table

1NF: All attributes are atomic (no multi-valued or composite attributes). Each row is uniquely identified by donor_id. The table is in 1NF.

2NF: The primary key is a single attribute (donor_id), so partial dependency is not possible. All non-key attributes — name, age, blood_group, phone, city, last_donated — are fully functionally dependent on donor_id. The table is in 2NF.

3NF: No transitive dependencies exist. None of the non-key attributes determine another non-key attribute (e.g., city does not determine blood_group). The table is in 3NF.

BCNF: The only functional dependency is donor_id → {all other attributes}. Since donor_id is the sole candidate key, the table is in BCNF.

Hospitals Table

1NF to BCNF: All attributes (hospital_id, name, city, contact) are atomic and single-valued. The sole determinant is hospital_id, which is the primary key. No partial or transitive dependencies exist. The table is in BCNF.

Blood_Inventory Table

1NF: All attributes are atomic. Each record is uniquely identified by inv_id. In 1NF.

2NF: Single-attribute primary key; no partial dependencies possible. In 2NF.

3NF: No transitive dependencies — blood_group does not determine units or vice versa; donor_id is a foreign key referencing DONORS, not a derivable attribute. In 3NF.

BCNF: The only functional dependency is inv_id → {blood_group, units, expiry_date, received_date, donor_id}. No non-trivial dependency from a non-superkey exists. In BCNF.

Blood_Requests Table

1NF: All attributes are atomic. priority and status use ENUM types (a single scalar value). In 1NF.

2NF: Single-attribute PK (req_id). All attributes fully depend on req_id. In 2NF.

3NF: hospital_id is a foreign key; it does not transitively determine patient, blood_group, or units. No transitive dependencies. In 3NF.

BCNF: The sole determinant for all attributes is req_id, the primary key. In BCNF.

Conclusion: All four tables of RaktSetu are in Boyce-Codd Normal Form (BCNF). The schema has no redundancy, no update anomalies, and no insertion or deletion anomalies.

VI. SQL Queries

All queries were run on **MySQL 8.0** using the raktsetu database. The schema was created using raktsetu.sql. Below are 22 queries covering all major SQL concepts.

Q1: Retrieve all donors

SELECT * FROM donors;

Purpose: Basic full-table retrieval. Confirms all 20 donor records are inserted correctly.

Q2: Retrieve all blood inventory records

SELECT * FROM blood_inventory;

Purpose: View the complete blood inventory (20 records across all blood groups).

Q3: Retrieve all hospitals

SELECT * FROM hospitals;

Purpose: Lists all 10 registered hospitals with their contact details.

Q4: Retrieve all pending blood requests

SELECT * FROM blood_requests WHERE status = 'PENDING';

Purpose: WHERE clause filter. Shows all requests still awaiting action — critical for daily operational review.

Q5: Find all O+ donors (WHERE filter)

SELECT name, phone, city FROM donors WHERE blood_group = 'O+';

Purpose: Targeted donor search. O+ is the most common blood group and useful for large-scale transfusions. *Expected output rows:* D001 (Arjun Sharma), D009 (Suresh Bhat), D013 (Rahul Verma)

Q6: Total units per blood group (GROUP BY + SUM)

```
SELECT blood_group, SUM(units) AS total_units

FROM blood_inventory

GROUP BY blood_group

ORDER BY total_units DESC;
```

Purpose: Aggregate function with GROUP BY. Gives a bird's-eye view of stock across blood groups. Useful for identifying surplus and deficit groups.

Q7: Donor count per blood group (GROUP BY + COUNT)

```
SELECT blood_group, COUNT(*) AS donor_count

FROM donors

GROUP BY blood_group

ORDER BY donor_count DESC;
```

Purpose: Reveals which blood groups have more registered donors, helping plan donation drives.

Q8: Average donor age per blood group (AVG)

```
SELECT blood_group, ROUND(AVG(age), 1) AS avg_age

FROM donors

GROUP BY blood_group;
```

Purpose: Statistical aggregate. Helps understand the demographic profile of donors for targeted health campaigns.

Q9: Donor details with their inventory contribution (INNER JOIN)

```
SELECT d.name, d.blood_group, bi.units, bi.expiry_date

FROM donors d

JOIN blood_inventory bi ON d.donor_id = bi.donor_id;
```

Purpose: INNER JOIN across DONORS and BLOOD_INVENTORY. Links each inventory batch back to the contributing donor.

Q10: Blood request details with hospital name (JOIN)

```
SELECT br.req_id, h.name AS hospital, br.patient,
    br.blood_group, br.units, br.priority, br.status
FROM blood_requests br
JOIN hospitals h ON br.hospital_id = h.hospital_id;
```

Purpose: Enriches blood request records with human-readable hospital names instead of just IDs.

Q11: Critical pending requests with hospital details (JOIN + WHERE)

```
SELECT br.req_id, h.name AS hospital, br.patient, br.blood_group, br.units
FROM blood_requests br
JOIN hospitals h ON br.hospital_id = h.hospital_id
WHERE br.priority = 'CRITICAL' AND br.status = 'PENDING';
```

Purpose: Emergency triage query. Identifies the most urgent requests that require immediate action.

Q12: Blood groups with total stock below 100 units (HAVING)

```
SELECT blood_group, SUM(units) AS total
FROM blood_inventory
GROUP BY blood_group
HAVING total < 100;
```

Purpose: HAVING clause on aggregated data. Flags critically low-stock blood groups for urgent procurement drives.

Q13: Donors who donated within the last 6 months

```
SELECT name, blood_group, last_donated

FROM donors

WHERE last_donated >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

Purpose: Date function usage. Identifies recently active donors who are eligible to donate again soon.

Q14: Inventory units expiring within the next 30 days

```
SELECT inv_id, blood_group, units, expiry_date

FROM blood_inventory

WHERE expiry_date <= DATE_ADD(CURDATE(), INTERVAL 30 DAY);
```

Purpose: Expiry alert query. Allows the blood bank to prioritise using or redistributing blood nearing expiry.

Q15: Total units requested per blood group

```
SELECT blood_group, SUM(units) AS units_requested

FROM blood_requests

GROUP BY blood_group

ORDER BY units_requested DESC;
```

Purpose: Demand-side aggregation. Helps the bank align procurement strategy with actual hospital demand.

Q16: Hospitals with more than one blood request (HAVING)

```
SELECT hospital_id, COUNT(*) AS total_requests

FROM blood_requests

GROUP BY hospital_id

HAVING total_requests > 1;
```

Purpose: Identifies hospitals that are frequent requestors — useful for establishing priority partnerships or dedicated supply agreements.

Q17: Donors from Pune

```
SELECT name, age, blood_group, phone

FROM donors

WHERE city = 'Pune';
```

Purpose: Location-based filter for organising city-level donor drives and quick local outreach.

Q18: UPDATE — Approve a pending request

```
UPDATE blood_requests

SET status = 'APPROVED'

WHERE req_id = 'R001';
```

Purpose: DML UPDATE statement. Demonstrates the status workflow transition from PENDING to APPROVED.

Q19: UPDATE — Dispense one unit from inventory

```
UPDATE blood_inventory

SET units = units - 1

WHERE inv_id = 'I001' AND units > 0;
```

Purpose: Safe decrement with a guard condition (units > 0) to prevent negative inventory. Mirrors the /api/inventory/<inv_id>/dispense API endpoint.

Q20: Fulfilled requests with hospital info (3-table JOIN)

```
SELECT br.req_id, h.name AS hospital, br.patient,

    br.blood_group, br.units, br.request_date

FROM blood_requests br
```

```
JOIN hospitals h ON br.hospital_id = h.hospital_id

WHERE br.status = 'FULFILLED';
```

Purpose: Multi-table JOIN with a WHERE filter. Provides a fulfilment history report showing which hospital requests have been successfully completed.

Q21: Inventory with donor city (LEFT JOIN)

```
SELECT bi.blood_group, bi.units, d.name AS donor_name, d.city

FROM blood_inventory bi

LEFT JOIN donors d ON bi.donor_id = d.donor_id

ORDER BY bi.blood_group;
```

Purpose: LEFT JOIN ensures all inventory records appear even if the associated donor has been deleted (donor_id becomes NULL). Demonstrates handling of optional relationships.

Q22: Inventory records with units above average (Subquery)

```
SELECT inv_id, blood_group, units

FROM blood_inventory

WHERE units > (SELECT AVG(units) FROM blood_inventory);
```

Purpose: Correlated subquery. Identifies blood group batches that are above the average stock level — useful for deciding where redistribution is feasible.

VII. Project Demonstration

Tools, Software, and Libraries Used

Category	Technology	Purpose
Database	MySQL 8.0	Relational database engine
Backend	Python 3.11, Flask	REST API server
CORS	flask-cors	Cross-origin request handling
DB Driver	mysql-connector-python	MySQL ↔ Python bridge
Frontend	HTML5, CSS3, JavaScript (Vanilla)	Web user interface
API Style	REST (JSON)	Communication between frontend and backend
Dev Tools	VS Code, MySQL Workbench	Development and DB management

System Architecture

The system follows a three-tier architecture:

Tier 1 — Presentation Layer: The index.html frontend is a single-page application divided into five panels: Dashboard, Donors, Inventory, Requests, and SQL Console. It communicates with the backend exclusively via fetch() API calls to REST endpoints.

Tier 2 — Application Layer: app.py (Flask) serves as the REST API. It defines 13 endpoints covering GET, POST, DELETE, and PATCH operations. Each endpoint opens a fresh MySQL connection using a helper function get_db(), executes the required SQL, and returns a JSON response. Date/datetime objects are serialised to ISO format strings via the rows_to_json() helper.

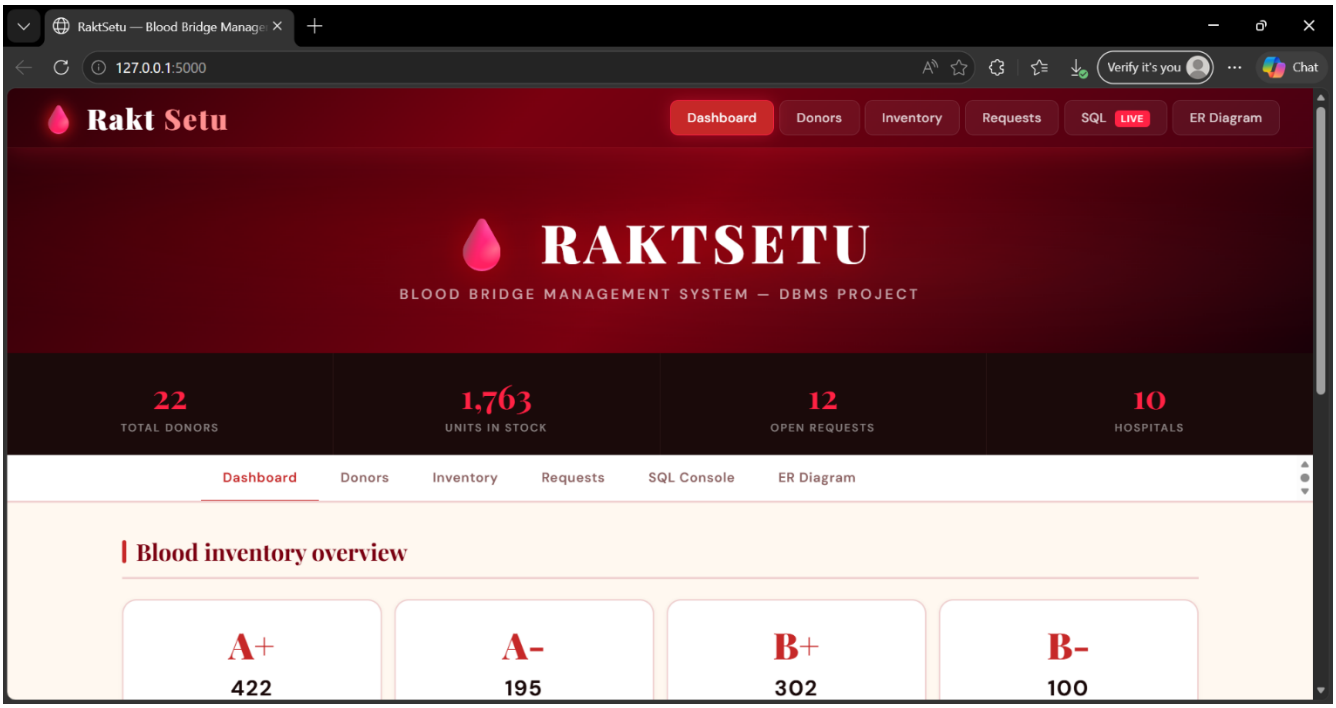
Tier 3 — Data Layer: The raktsetu MySQL database contains four tables (donors, hospitals, blood_inventory, blood_requests) with foreign key constraints and three performance indexes.

REST API Endpoints

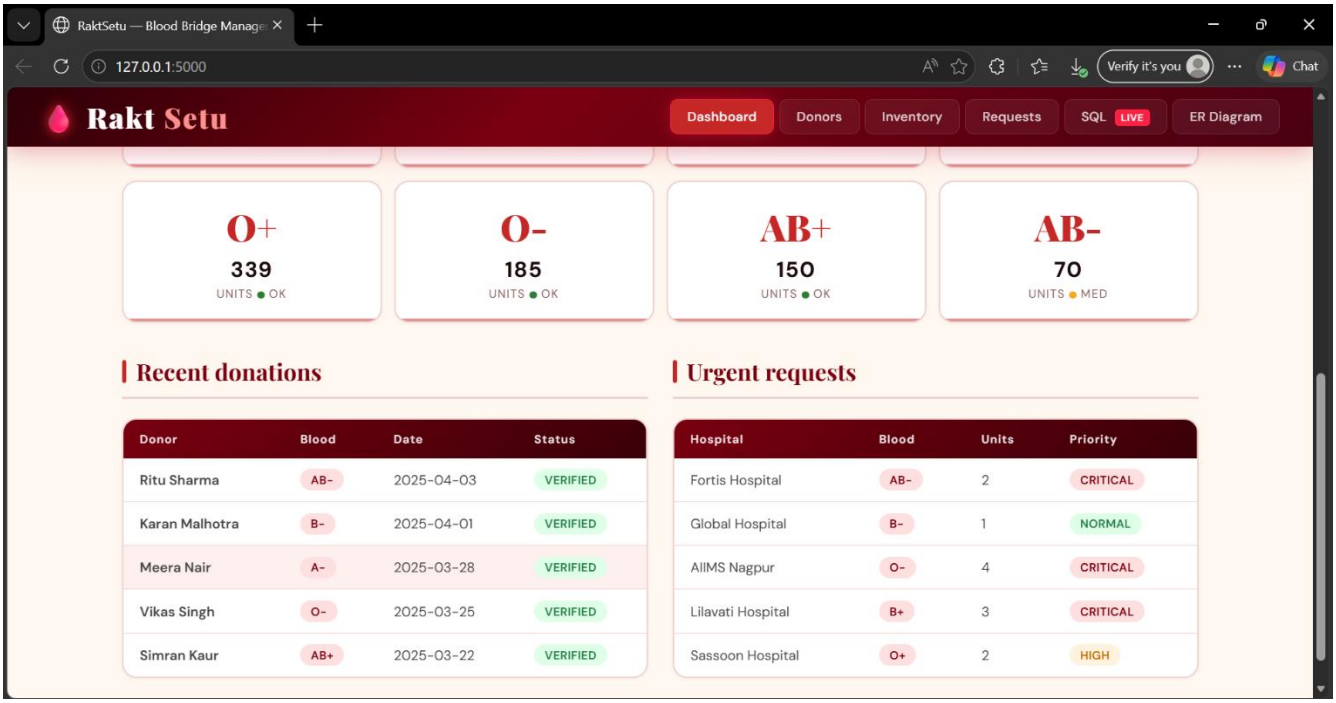
Method	Endpoint	Description
GET	/api/stats	Dashboard summary (donor count, total units, pending requests, hospital count)
GET	/api/donors	List donors with optional search and blood group filter
POST	/api/donors	Register a new donor
DELETE	/api/donors/<donor_id>	Remove a donor
GET	/api/inventory	Full inventory listing
GET	/api/inventory/summary	Aggregated units per blood group
POST	/api/inventory	Add new blood units to inventory
POST	/api/inventory/<inv_id>/dispense	Decrement one unit from a batch
GET	/api/requests	List all blood requests (with hospital names via JOIN)
POST	/api/requests	Submit a new blood request
PATCH	/api/requests/<req_id>/status	Update request status to APPROVED or FULFILLED
GET	/api/hospitals	List all hospitals
POST	/api/sql	Execute a custom SQL query (SELECT/INSERT/UPDATE/DELETE)

Screenshot Descriptions

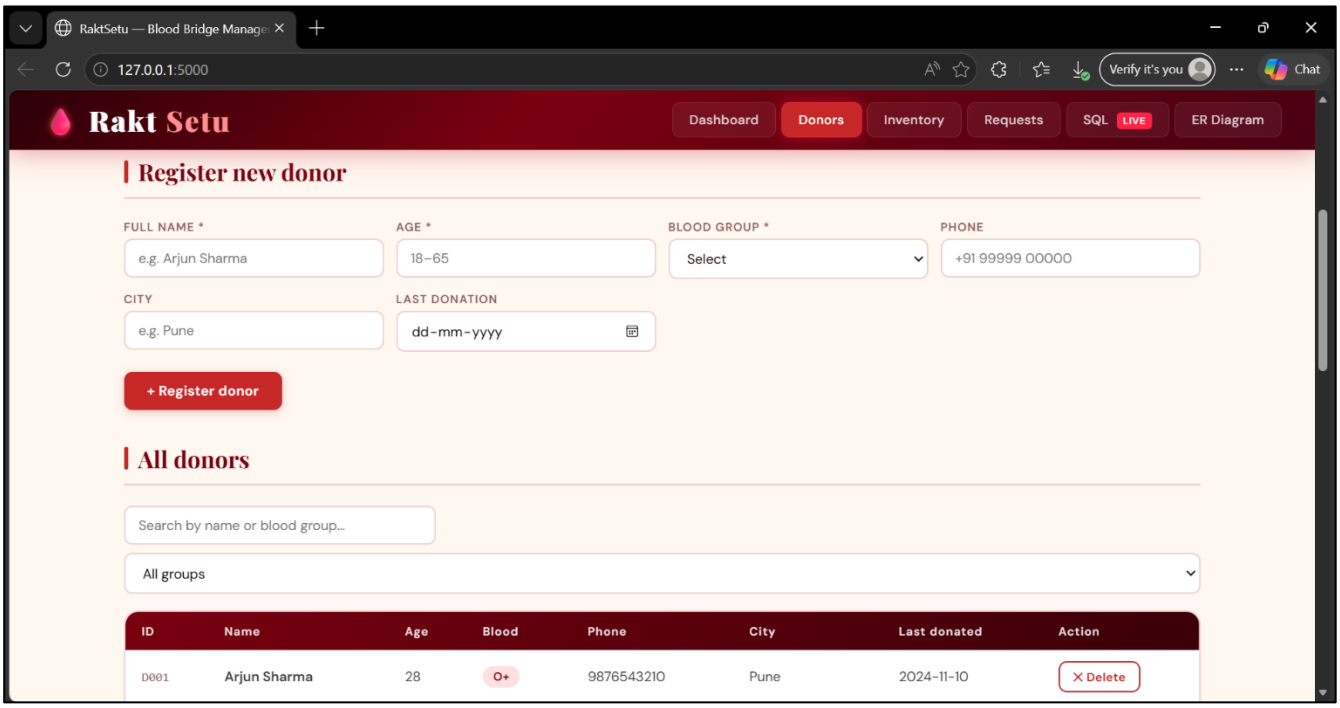
Dashboard Panel: Displays four live counters Total Donors, Units in Stock, Pending Requests, and Hospitals. These are fetched from /api/stats on page load.



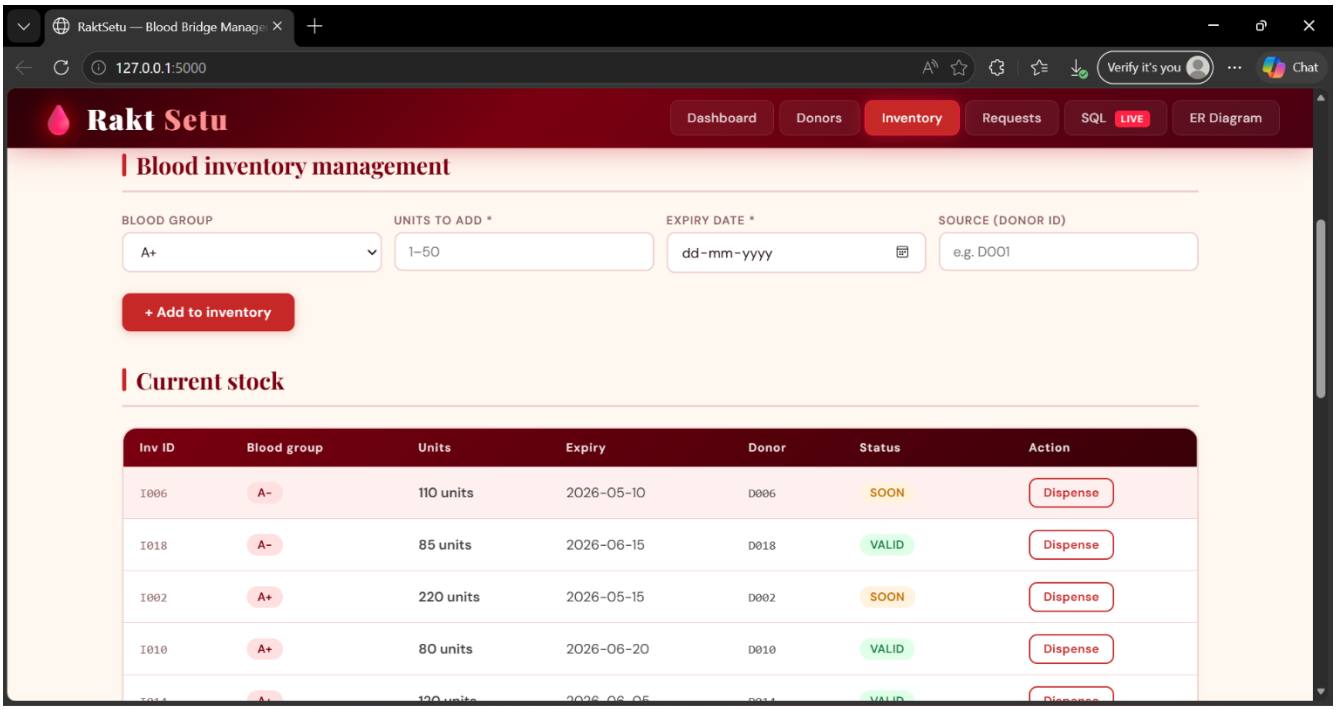
Section for Recent Donations and Urgent Requests where top 5 are displayed



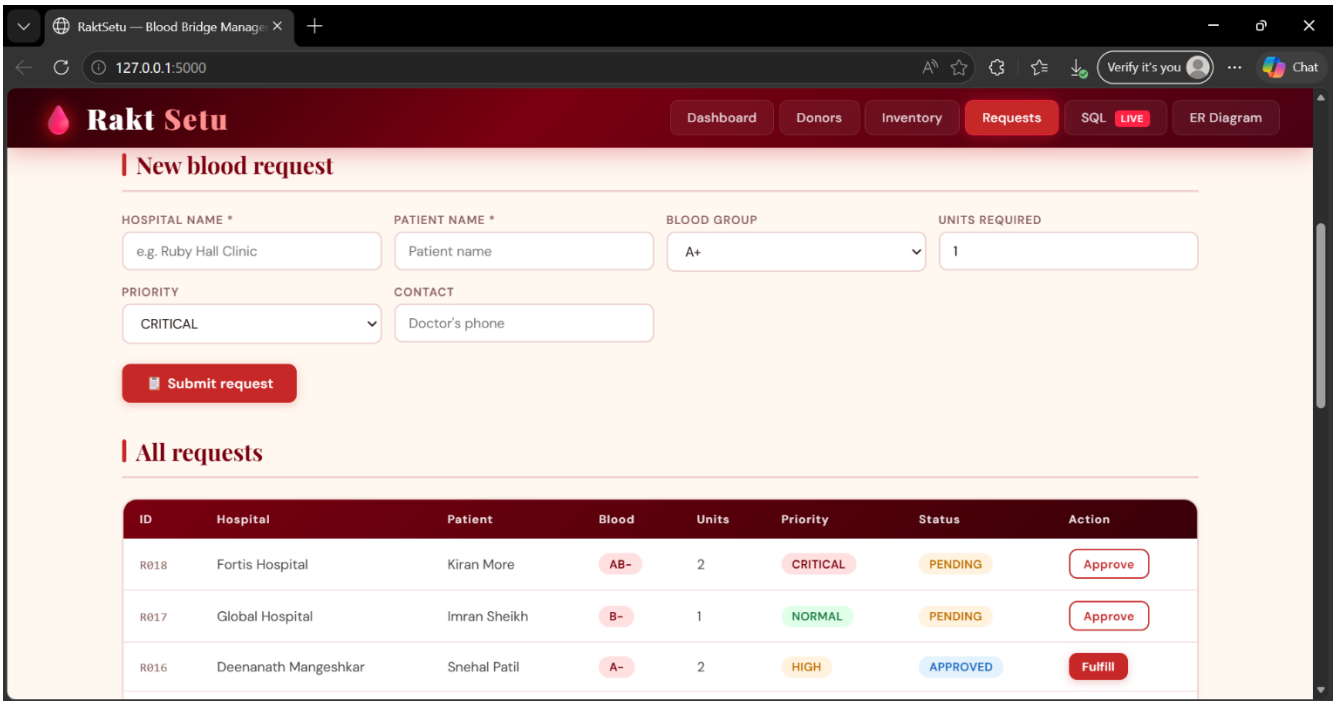
Donors Panel: Shows a searchable, filterable table of all registered donors. A form at the top allows adding new donors. A delete button per row calls the DELETE endpoint.



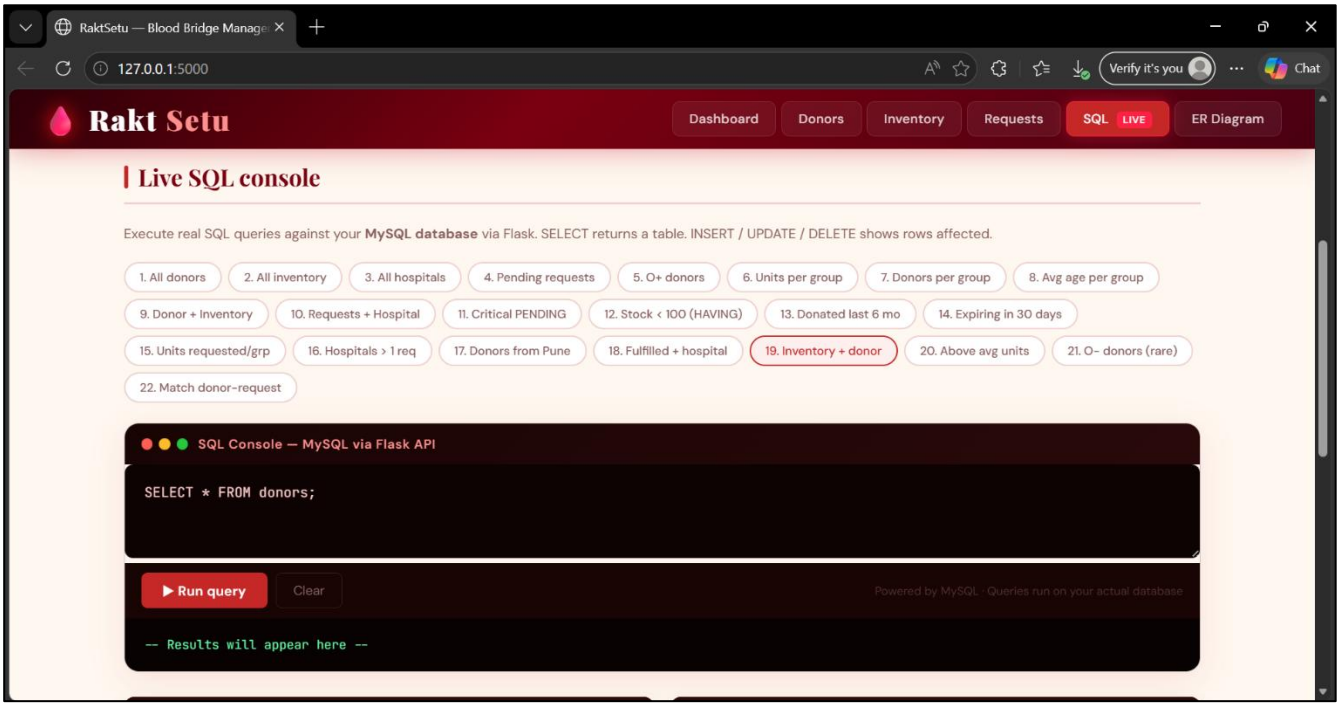
Inventory Panel: Displays a summary chart of units per blood group alongside a detailed table with expiry dates. An "Add Inventory" form and per-row "Dispense" button allow stock management.



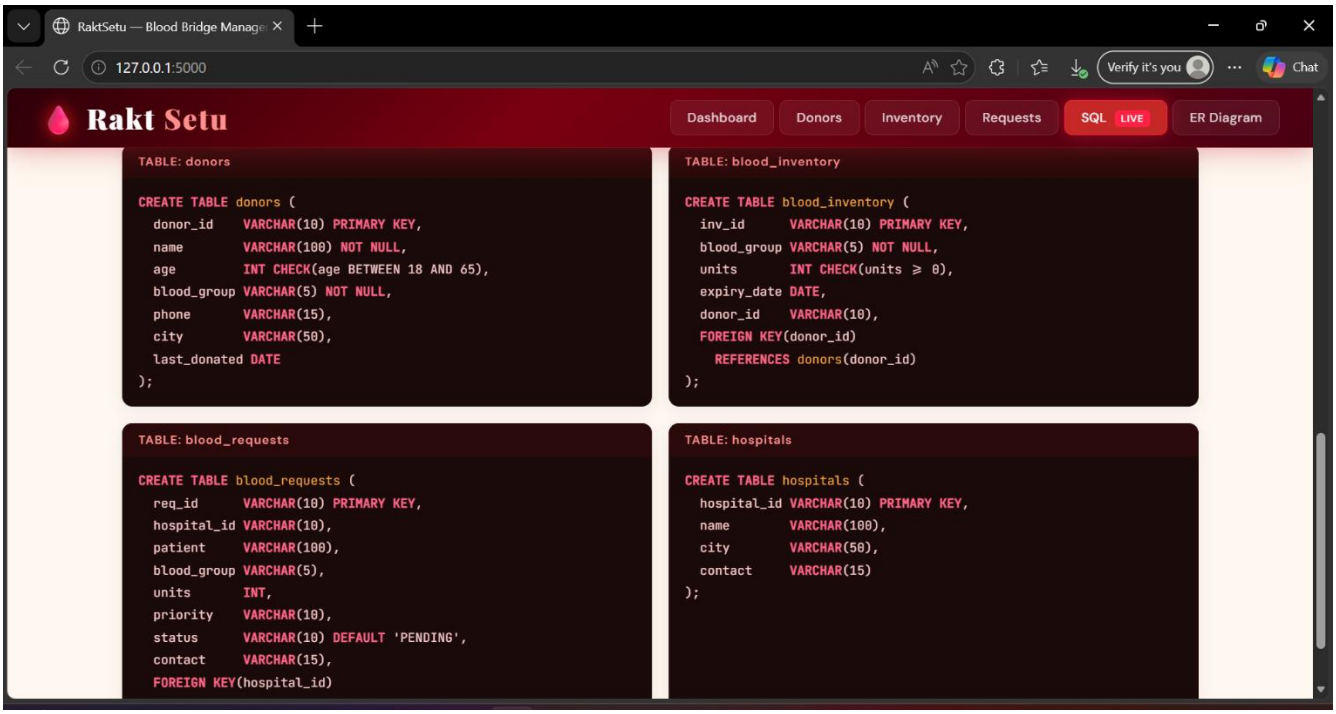
Requests Panel: Lists all blood requests with priority colour coding (red for CRITICAL, orange for HIGH, green for NORMAL). Status can be updated per row.



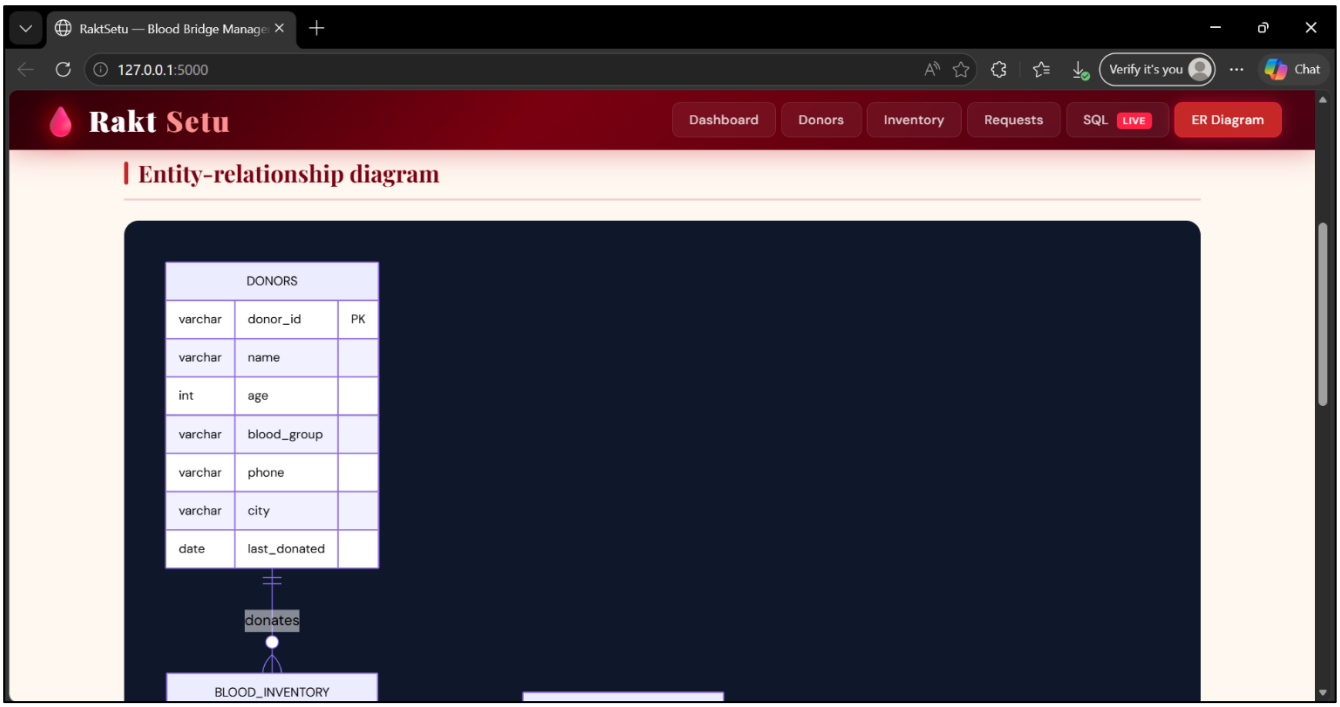
SQL Console Panel: An in-app SQL terminal allowing operators to type and execute any SELECT, INSERT, UPDATE, or DELETE query. Results are displayed in a dynamic table or as a row-count message.

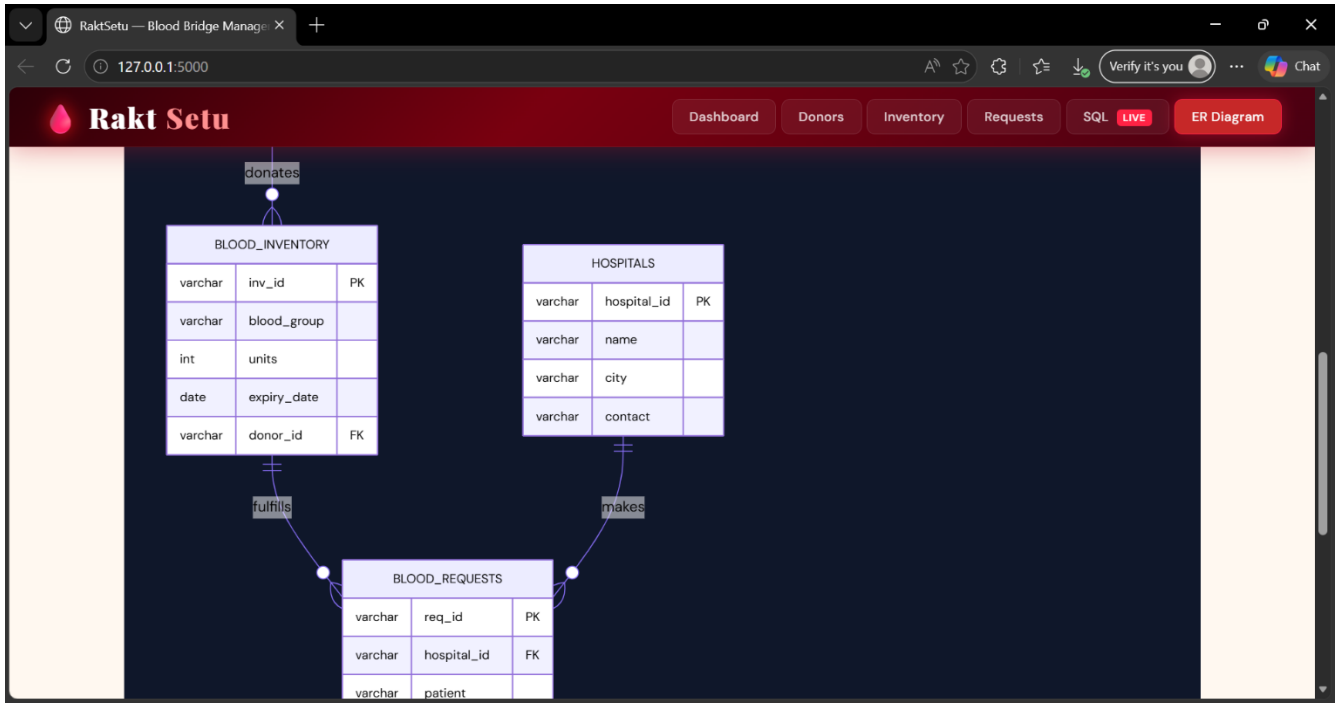


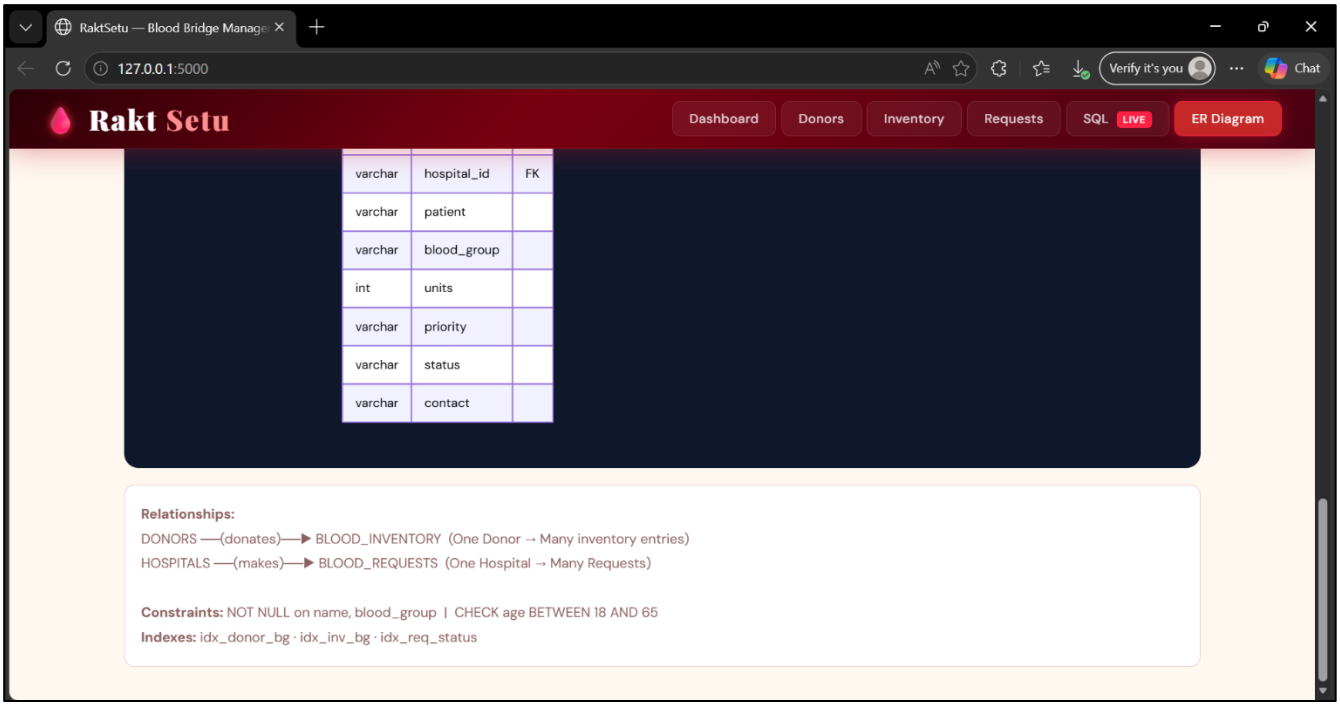
Data Tables displayed on the SQL live tab



ER Diagram: Entity Relationship Diagram showing how the tables are connected with relationships, constraints and indexes mentioned




RaktSetu — Blood Bridge Manage
127.0.0.1:5000
Verify it's you
Chat
Rakt Setu
Dashboard
Donors
Inventory
Requests
SQL LIVE
ER Diagram
donates
BLOOD_INVENTORY
varchar inv_id PK
varchar blood_group
int units
date expiry_date
varchar donor_id FK
HOSPITALS
varchar hospital_id PK
varchar name
varchar city
varchar contact
fulfills
makes
BLOOD_REQUESTS
varchar req_id PK
varchar hospital_id FK
varchar patient


RaktSetu — Blood Bridge Manage
127.0.0.1:5000
Verify it's you
Chat
Rakt Setu
Dashboard
Donors
Inventory
Requests
SQL LIVE
ER Diagram
varchar hospital_id FK
varchar patient
varchar blood_group
int units
varchar priority
varchar status
varchar contact
Relationships:
DONORS —(donates)—▶ BLOOD_INVENTORY (One Donor → Many inventory entries)
HOSPITALS —(makes)—▶ BLOOD_REQUESTS (One Hospital → Many Requests)
Constraints: NOT NULL on name, blood_group | CHECK age BETWEEN 18 AND 65
Indexes: idx_donor_bg · idx_inv_bg · idx_req_status

VIII. Self-Learning Beyond Classroom

The following topics were learned independently, going beyond the DBMS course curriculum:

1. REST API Design with Flask (Varshita Vaan) The DBMS course covers SQL and relational theory but not API development. We independently learned how Flask routes map to HTTP methods (GET, POST, PATCH, DELETE), how to parse JSON request bodies, and how to return structured JSON responses. We also learned about CORS (Cross-Origin Resource Sharing) and why it is necessary when a frontend served at one origin communicates with a backend at another.

2. Dynamic ID Generation in SQL (Yash Shah) Rather than using MySQL's AUTO_INCREMENT, we implemented application-level ID generation (D001, D002, etc.) in Python. This required understanding how to query the last inserted ID, increment it safely, and handle collision cases in a loop a pattern commonly used in legacy systems that need human-readable primary keys.

3. Single-Page Application (SPA) Architecture (Mahak Wadhawan) The entire frontend is built as a single HTML file with JavaScript-driven panel switching, rather than multiple separate pages. We learned how to use the Fetch API, async/await, dynamic DOM manipulation, and event-driven design to build a responsive interface without any frontend frameworks.

4. MySQL Date Functions We independently explored functions such as DATE_SUB(), DATE_ADD(), CURDATE(), and INTERVAL expressions used in Queries 13 and 14 which go beyond the basic SQL taught in class.

5. Database Connection Pooling and Exception Handling We learned that opening and closing a database connection per request (as implemented in our get_db() helper) is adequate for a development system, and studied how production systems use connection pooling (via libraries like SQLAlchemy) to handle concurrent users efficiently.

IX. Learning from the Project

1. The value of relational integrity: Implementing foreign keys (ON DELETE SET NULL) taught us practically why constraints matter. When we deleted a donor during testing, we saw first-hand that the inventory records retained their data with a NULL donor_id rather than being orphaned or corrupted.

2. Separating concerns: Building a three-tier system (frontend, Flask API, MySQL) reinforced the principle of separation of concerns. The frontend never directly queries MySQL — it only speaks JSON with the API layer. This made debugging much easier since each tier could be tested independently.

3. SQL is more powerful than we assumed: Before this project, we primarily used SQL for simple SELECT queries. Building 22 queries covering JOINs, subqueries, HAVING, date functions, and DML operations showed us how much analytical and operational work can be done at the database layer without writing application code.

4. Normalisation has real consequences: Normalising to BCNF meant we never stored a hospital's name inside the blood_requests table. Initially this felt restrictive, but it taught us why redundancy causes update anomalies if a hospital changes its name, we update exactly one row in HOSPITALS rather than dozens in BLOOD_REQUESTS.

5. Teamwork and version control discipline: Dividing the project into frontend, backend, and database layers each owned by a different team member required clear interface contracts (API schemas, table structures) agreed upon upfront. This mirrors real-world software development practices.

X. Challenges Faced

1. Date serialisation in Flask: MySQL's Python connector returns DATE columns as Python datetime.date objects, which Flask's jsonify() cannot serialise by default. This caused 500 errors on every API call that returned date data. We resolved this by writing a custom rows_to_json() helper that converts date objects to ISO format strings before returning them.

2. Flask route ordering conflict: The endpoint /api/inventory/summary conflicted with /api/inventory/<inv_id>/dispense because Flask was interpreting "summary" as an inv_id variable. We resolved this by placing the literal route /api/inventory/summary above the variable route in the code, as Flask matches routes in definition order.

3. Generating unique IDs safely: We wanted human-readable IDs (D001, H001, etc.) instead of numeric AUTO_INCREMENT. The challenge was avoiding duplicates in a concurrent environment. We implemented a check-and-increment loop that queries for an existing ID before committing — adequate for a single-user system.

4. CORS errors during development: When testing the frontend directly from the filesystem (file:// protocol), browsers blocked API calls to the Flask server due to CORS policy. We resolved this by adding the flask-cors library and enabling CORS globally for the development environment.

5. Maintaining referential integrity with realistic seed data: Populating 20 donors, 10 hospitals, 20 inventory records, and 18 blood requests while ensuring all foreign keys were valid required careful ordering of INSERT statements. We had to insert DONORS and HOSPITALS before BLOOD_INVENTORY and BLOOD_REQUESTS respectively, and cross-check every donor_id and hospital_id reference manually.

XI. Conclusion

RaktSetu — Blood Bridge Management System successfully demonstrates how a well-designed relational database can serve as the foundation for a real-world healthcare application. Through this project, we built a complete, working system spanning database design, SQL implementation, a Python REST API, and a web frontend.

Key Takeaways:

The project reinforced that good database design proper normalisation, foreign key constraints, and indexing is not academic theory but a practical necessity that prevents data corruption and improves system performance. Writing 22+ SQL queries covering every major concept (DDL, DML, JOINs, subqueries, aggregate functions, date functions, HAVING clauses) gave us genuine fluency in SQL that goes beyond classroom exercises.

From a systems perspective, we learned that three-tier architecture cleanly separates responsibilities: the database handles data integrity, the API handles business logic, and the frontend handles user experience. This separation made our system easier to debug, test, and extend.

The domain of blood management is personally meaningful inefficiency in this area costs lives. Building RaktSetu gave us a sense of how technology can solve real human problems, which is ultimately why we study computer engineering.

We are confident that the skills learned here relational modelling, SQL, API development, and frontend integration will serve as a strong foundation for advanced coursework and professional software development.